

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Eya Damak GI2S4

Année Univeristaire : 2025-2026

Contents

1	Contexte & Donnees	2
1.1	Problematique	2
1.2	Objectifs du projet	2
1.3	Description du jeu de données	2
1.4	Déséquilibre des classes	3
1.5	Diagnostic de la qualité des données	3
1.6	Diagnostic qualite des donnees	4
1.7	Analyse des corrélations	4
2	Pipeline de Traitement	5
2.1	Nettoyage initial	5
2.2	Feature engineering	6
2.3	Détection et correction du data leakage	6
2.3.1	Définition et impact	6
2.3.2	Identification du problème	6
2.3.3	Correction et résultats	6
2.3.4	Colonnes supprimées	7
2.4	Encodage des variables catégorielles	7
2.5	Séparation des données et normalisation	7
3	Apprentissage Non Supervise : ACP + Clustering	8
3.1	ACP non supervisé	8
3.1.1	Résultats	8
3.2	Clustering non supervisé — K-Means	9
3.2.1	Choix du nombre de clusters	9
3.2.2	Présentation des segments clients	10
4	Modèles de Classification Supervisée du Churn	10
4.1	Random Forest	10
4.2	XGBoost	12
4.3	Stacking (Random Forest + XGBoost)	13
4.4	Analyse de l'importance des features	15
5	Prédiction de la Valeur Client (Régression)	16
5.1	Objectif	16
5.2	XGBoost Regressor	16
6	Performance & Analyse Comparative	17
6.1	Comparaison globale des modèles de classification	17
6.2	Robustesse du modèle : correction du data leakage	17
7	Application Web :le déploiemenet	18
7.1	Architecture de l'application Flask	18
7.2	Fonctionnalités de l'interface	18
	Conclusion	21

1 Contexte & Donnees

1.1 Problematique

Dans un environnement e-commerce caractérisé par une concurrence accrue, la fidélisation de la clientèle constitue un enjeu stratégique majeur. En effet, le coût d'acquisition d'un nouveau client est généralement estimé entre cinq et sept fois supérieur à celui de la rétention d'un client existant. Dans ce contexte, l'entreprise de vente en ligne de cadeaux étudiée vise à anticiper les départs de ses clients — phénomène communément désigné par le terme **churn** — afin de mettre en œuvre des actions de fidélisation proactives, ciblées et efficaces, en s'appuyant sur l'analyse des données transactionnelles et comportementales.

1.2 Objectifs du projet

Ce projet s'inscrit dans une démarche de data science appliquée et poursuit trois objectifs méthodologiques interdépendants :

1. **Segmentation de la clientèle** : identification de groupes homogènes de clients à l'aide de techniques de clustering non supervisé, notamment l'algorithme K-Means appliqué sur les composantes issues de l'Analyse en Composantes Principales (ACP).
2. **Prédiction du churn** : développement de modèles de classification supervisée (Random Forest, XGBoost et Stacking) visant à détecter les clients présentant un risque élevé de désengagement avec un niveau de performance optimal.
3. **Estimation de la valeur client** : modélisation de la valeur économique des clients par des techniques de régression, en particulier via un XGBoost Regressor, afin de hiérarchiser les actions marketing en fonction de l'impact financier potentiel.

1.3 Description du jeu de données

Le jeu de données étudié, `retail_customers_COMPLETE_CATEGORICAL.csv`, constitue la base principale de ce travail. Il comprend 4372 clients décrits à travers 52 variables. Ces variables couvrent différentes dimensions, notamment les comportements d'achat (fréquence, montant, récence), les préférences clients, les caractéristiques démographiques ainsi que des indicateurs de satisfaction. Afin de reproduire des conditions proches du monde réel, ce jeu de données contient volontairement certaines anomalies.

Table 1: Caractéristiques générales du jeu de données

Caractéristique	Valeur
Nombre de clients	4 372
Nombre de variables	52
Variable cible	Churn (0 : client actif, 1 : client perdu)
Répartition des classes	67% actifs / 33% churners
Variables numériques	34
Variables catégorielles	18

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	CustomerID	Recency	Frequency	MonetaryTotal	MonetaryAvg	MonetaryStd	MonetaryMin	MonetaryMax	TotalQuantity	AvgQuantityPer	MinQuantity	MaxQuantity	CustomerTenure	FirstPurchaseDe	PreferredC
2	17850	302	35	5288.63000000	16.950731794	13.60366168911	-30.6	107.25	1693	5.42628205128	-12	12	71	374	
3	13047	32	18	3079.1	15.7096938775	11.68476882396	-15.0	68.0	1355	6.91326530612	-3	32	342	374	
4	12583	3	18	17187.34	28.6348207171	23.15013187581	-60.84	132.8	5009	19.9561752988	-36	360	370	374	
5	13748	96	5	948.25	33.8660714285	42.95311926234	9.36	204.0	439	15.6785714285	1	80	278	374	
6	15100	330	6	635.1	105.85	215.9862634521	-131.399999999	350.4	58	9.66666666666	-12	32	43	374	
7	15291	26	20	4596.51	42.1698165137	49.0906065611	-35.49	243.36	2074	19.0275229357	-21	144	348	374	
8	14688	8	27	5107.38	14.2266852367	23.6444697820	-72.15	153.0	3222	8.97493036211	-39	120	366	374	
9	17809	16	15	4627.62	72.3065625	142.206440338	-717.23	425.0	2015	31.484375	-40	144	357	374	
10	15311	1	118	59419.34	23.8536089623	46.6250773177	-275.0	408.0	37720	15.1425130469	-100	288	373	374	
11	14527	3	88	7711.38	7.62747774480	9.75929138741	-80.0	87.6	2049	2.02670823145	-2	36	371	374	
12	16098	88	7	2005.63	29.6347761194	35.4309518580	3.75	178.200000000	613	9.14925373134	1	50	286	374	
13	18074	374	1	489.6	37.6615384615	28.2543739522		15.8	190	14.6153846153	2	48	0	374	
14	17420	50	3	598.83	19.961	5.9650460326	12.48	39.6	265	8.63333333333	1	24	323	374	
15	16029	39	78	50992.61	186.104416058	933.772349059	-8142.75	8142.75	32148	117.3284671532	-1296	2400	335	374	
16	16250	261	2	389.44	16.2266666666	4.73140999911		05.04	208	8.66666666666	2	24	112	374	
17	12431	36	18	6416.38999999	26.7349583333	50.2015987426	-38.16	601.44	2868	11.95	-36	336	338	374	
18	17511	3	46	88125.38	81.9009107806	87.9782893224	-179.0	745.0	63012	58.5613382896	-100	600	370	374	
19	17548	218	3	-141.48	-8.32235294117	21.7951518489	-41.4000000000	30.0	-132	-7.76470588235	-24	24	155	374	
20	13705	8	3	711.86	25.4235714285	29.7731293487	7.80000000000	165.0	363	12.9642857142	1	32	366	374	
21	13747	374	1	79.6	79.6	0.0	79.6	79.6	8	8.0	8	8	0	374	
22	13408	2	81	27487.41	54.8650898203	50.5407405509	-110.43	306.0	16119	32.1736526946	-48	192	371	374	
23	13767	2	52	10945.71	42.4704511278	15.9851746779	-63.75	532.800000000	7258	18.1904761904	-11	288	371	374	
24	17924	1	13	2894.33	76.1665789473	82.6811853912	-13.95	367.200000000	1141	30.0263157894	-5	120	372	374	
25	13448	17	10	3465.67000000	17.4154271356	13.90786399761	-29.75	60.0	1638	8.231155778894	-12	48	357	374	

Figure 1: Aperçu du jeu de données retail_customers_COMPLETE_CATEGORICAL

1.4 Déséquilibre des classes

L'analyse de la variable cible *Churn* met en évidence un déséquilibre significatif des classes. En effet, la distribution des observations montre qu'environ 67% des clients sont actifs contre 33% de clients churners, soit un ratio proche de 2:1.

Ce **déséquilibre des classes** constitue un enjeu majeur lors de la phase de modélisation, car il peut introduire un biais en faveur de la classe majoritaire et réduire la capacité des modèles à détecter correctement les clients à risque.

Afin de remédier à ce problème, une technique de rééchantillonnage a été utilisée, à savoir le *SMOTE* (Synthetic Minority Oversampling Technique), qui permet de générer des exemples synthétiques de la classe minoritaire afin d'équilibrer le jeu de données et d'améliorer la qualité de l'apprentissage.

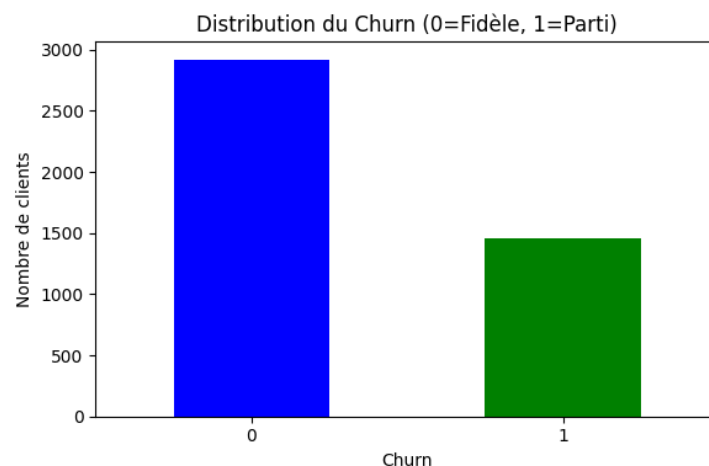


Figure 2: Distribution de la variable Churn (67% clients actifs vs 33% churners)

1.5 Diagnostic de la qualité des données

Une étape essentielle de l'analyse exploratoire consiste à évaluer la qualité du jeu de données afin d'identifier les éventuelles anomalies susceptibles d'affecter les performances des modèles. Plusieurs types de problèmes ont été détectés, allant des valeurs manquantes aux problèmes de multicollinéarité, en passant par des incohérences de format et des variables peu informatives.

1.6 Diagnostic qualite des donnees

L'étape de diagnostic de la qualité des données constitue une phase fondamentale dans tout projet de data science. Elle permet d'identifier les anomalies, incohérences et limitations du jeu de données, susceptibles d'influencer négativement les performances des modèles d'apprentissage. Dans ce cadre, plusieurs problématiques ont été détectées et traitées, notamment les valeurs manquantes, les valeurs aberrantes, les problèmes de format, la multicolinéarité ainsi que le déséquilibre des classes. Le tableau ci-dessous synthétise l'ensemble des problèmes identifiés ainsi que les traitements appliqués pour y remédier.

Table 2: Synthèse des problèmes de qualité des données et des traitements appliqués

Type de problème	Variables concernées	Ampleur	Traitement appliqué
Valeurs manquantes	Age, AvgDaysBetweenPurchases	30%	Imputation par la médiane sur l'ensemble d'entraînement
Valeurs aberrantes	SupportTickets, SatisfactionScore	5–8%	Remplacement par NaN puis imputation
Formats inconsistants	RegistrationDate	100%	Standardisation via <code>pd.to_datetime(dayfirst=True)</code>
Variance nulle	NewsletterSubscribed	100%	Suppression de la variable
Données brutes non structurées	LastLoginIP	100%	Transformation en variable binaire <i>IsPrivateIP</i>
Déséquilibre de classes	Churn	—	Rééchantillonnage par <i>SMOTE</i> (<code>sampling_strategy = 0.5</code>)
Multicolinéarité	MonetaryMin, MonetaryStd, etc.	$r > 0.8$	Suppression basée sur l'analyse VIF

1.7 Analyse des corrélations

L'analyse de la matrice de corrélation appliquée aux variables numériques permet de mettre en évidence plusieurs relations linéaires fortes entre certaines caractéristiques du jeu de données. Ces corrélations indiquent une redondance potentielle de l'information, pouvant conduire à des problèmes de multicolinéarité lors de la phase de modélisation.

Les principales paires fortement corrélées identifiées sont les suivantes :

- $\text{MonetaryMin} \leftrightarrow \text{MonetaryStd} : r = 0,97$
- $\text{TotalQuantity} \leftrightarrow \text{AvgQtyPerTrans} : r = 0,91$

- $\text{UniqueProducts} \leftrightarrow \text{UniqueDesc} : r = 0,88$

Par ailleurs, l'analyse du *Variance Inflation Factor (VIF)* confirme la présence d'une multicolinéarité élevée ($VIF > 10$) pour plusieurs variables à forte composante monétaire, ce qui justifie leur suppression afin de stabiliser les modèles et d'améliorer leur généralisation.

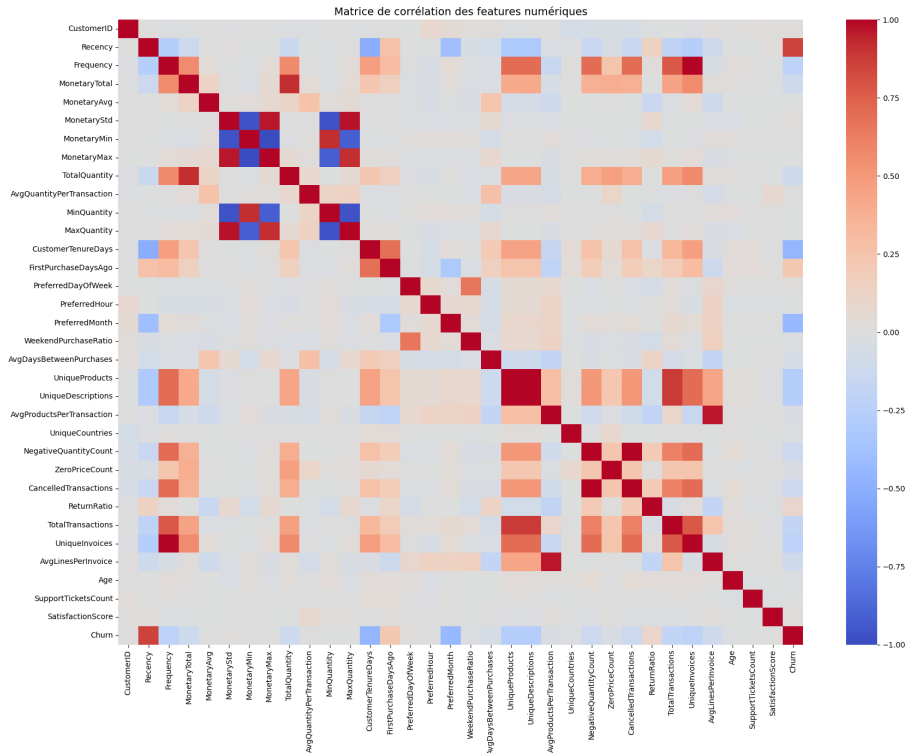


Figure 3: Matrice de corrélation des variables numériques du jeu de données

2 Pipeline de Traitement

Le pipeline de preparation est implemente dans `preprocessing.py` et suit une sequence stricte garantissant l'absence de fuite d'information entre train et test.

2.1 Nettoyage initial

La phase de nettoyage initial constitue une étape essentielle du prétraitement des données, visant à améliorer la qualité et la cohérence du jeu de données avant la modélisation. Plusieurs opérations de correction et de transformation ont été réalisées :

1. **Suppression des variables non informatives** : les attributs `CustomerID` et `NewsletterSubscribed` ont été supprimés en raison de leur absence de variabilité et de leur faible contribution explicative.
2. **Traitement des valeurs aberrantes** : les valeurs sentinelles (-1 , 999 pour `SupportTickets` et -1 , 99 pour `SatisfactionScore`) ont été considérées comme des données manquantes et remplacées par `NaN`.
3. **Standardisation des variables temporelles** : la variable `RegistrationDate`, initialement hétérogène en termes de formats, a été normalisée par parsing multi-format, puis décomposée en plusieurs attributs temporels (`RegYear`, `RegMonth`, `RegDay`, `RegWeekday`).

4. **Transformation de variables réseau** : la variable `LastLoginIP` a été convertie en variable binaire `IsPrivateIP`, indiquant si l'adresse IP appartient à une plage privée conforme à la norme RFC 1918.

2.2 Feature engineering

Dans le but d'enrichir l'information disponible et de mieux capturer les comportements clients, plusieurs variables dérivées ont été construites à partir des attributs existants. Ces nouvelles variables traduisent des ratios métier pertinents permettant une meilleure interprétation des dynamiques d'achat.

$$\text{MonetaryPerDay} = \frac{\text{MonetaryTotal}}{\text{Recency} + 1} \quad (1)$$

$$\text{AvgBasketValue} = \frac{\text{MonetaryTotal}}{\text{Frequency}} \quad (2)$$

$$\text{TenureRatio} = \frac{\text{Recency}}{\text{CustomerTenureDays} + 1} \quad (3)$$

Une analyse complémentaire a montré que la variable `TenureRatio`, bien que construite dans une logique exploratoire, présentait une corrélation élevée avec la variable cible *Churn* ($r = 0,86$). En raison de ce risque de *data leakage*, cette variable a été supprimée lors de la phase de préparation des données afin de garantir la robustesse des modèles.

2.3 Détection et correction du data leakage

2.3.1 Définition et impact

Le *data leakage* désigne une situation dans laquelle des variables utilisées pour l'apprentissage contiennent, de manière directe ou indirecte, des informations issues de la variable cible. Ce phénomène introduit un biais majeur, conduisant le modèle à apprendre des relations non généralisables.

Ainsi, bien que les performances en phase de validation puissent sembler exceptionnellement élevées, celles-ci ne reflètent pas la capacité réelle du modèle à généraliser sur de nouvelles données.

2.3.2 Identification du problème

Lors des premières expérimentations, le modèle Random Forest obtenait une accuracy de **100 %**, un résultat anormalement élevé et peu réaliste dans un contexte de données réelles. Cette observation a mis en évidence la présence probable d'une fuite de données.

2.3.3 Correction et résultats

Après analyse approfondie et suppression des variables responsables de ce biais, une nouvelle évaluation du modèle a été réalisée. L'accuracy a alors été ramenée à **95,2 %**, une performance plus réaliste et cohérente avec la complexité du problème. Cette correction a permis de garantir la robustesse et la capacité de généralisation du modèle.

2.3.4 Colonnes supprimées

Dans le cadre de la détection et de la correction du data leakage, une analyse approfondie des variables a été menée afin d'identifier les attributs susceptibles d'introduire un biais dans les modèles de prédiction. Cette étape a permis de mettre en évidence plusieurs catégories de variables problématiques, soit en raison de leur construction directe à partir de la variable cible, soit en raison d'une corrélation excessive ou de leur indisponibilité en contexte réel. Ces variables ont par conséquent été supprimées afin de garantir la robustesse et la généralisabilité des modèles.

Table 3: Synthèse des variables supprimées pour cause de data leakage

Catégorie	Variables supprimées
Variables construites à partir de la cible	ChurnRiskCategory, CustomerType, LoyaltyLevel, SpendingCategory, RFMSegment, AccountStatus
Variables comportementales post-événement	ReturnRatio, NegQtyCount, ZeroPriceCount, CancelledTransactions
Variables fortement corrélées à la cible	CustomerTenureDays, FirstPurchase, Age
Variables à forte proportion d'anomalies	SupportTicketsCount, SatisfactionScore
Variables en corrélation directe ($r > 0,85$)	Recency ($r = 0,86$), TenureRatio

Par ailleurs, un mécanisme de contrôle automatique a été intégré dans le script de prétraitement `preprocessing.py`. Ce mécanisme vérifie systématiquement les corrélations entre chaque variable et la cible *Churn*. Toute variable présentant une corrélation supérieure à 0,85 déclenche une alerte et entraîne l'arrêt du processus afin d'éviter toute fuite d'information.

2.4 Encodage des variables catégorielles

Afin de rendre les variables qualitatives exploitables par les algorithmes de machine learning, plusieurs techniques d'encodage ont été appliquées en fonction de la nature et de la cardinalité des attributs.

- **Encodage ordinal** : utilisé pour les variables présentant un ordre implicite, notamment *AgeCategory*, *BasketSizeCategory* et *PreferredTimeOfDay*.
- **Encodage One-Hot** : appliqué aux variables sans relation d'ordre, telles que *FavoriteSeason*, *Region*, *WeekendPreference*, *ProductDiversity* et *Gender*.
- **Encodage One-Hot différé pour Country** : en raison de sa forte cardinalité (plus de 37 modalités), cette variable est encodée après la séparation train/test afin d'éviter toute fuite d'information, avec le paramètre `handle_unknown='ignore'`.

2.5 Séparation des données et normalisation

Cette étape vise à préparer les données pour l'apprentissage tout en garantissant une évaluation fiable et sans biais du modèle.

- **Séparation stratifiée (80/20) :** les données sont divisées en ensembles d'entraînement et de test en conservant la distribution de la variable cible (`random_state=42`), soit 3 497 observations pour l'entraînement et 875 pour le test.
- **Imputation des valeurs manquantes :** réalisée via la médiane de chaque colonne, calculée uniquement sur les données d'entraînement (`X_train.median()`) puis appliquée à `X_train` et `X_test` via `fillna()`, afin d'éviter toute fuite d'information.
- **Normalisation :** standardisation des variables numériques via `StandardScaler`, ajusté sur `X_train` puis appliqué à `X_test`.

À l'issue de ces opérations de prétraitement, incluant les encodages catégoriels (ordinal, One-Hot, et One-Hot différé pour `Country` — effectué après la séparation pour éviter toute fuite d'information), le jeu de données final est composé de **87 variables explicatives**.

3 Apprentissage Non Supervisé : ACP + Clustering

3.1 ACP non supervisé

3.1.1 Résultats

L'ACP a été appliquée sur les données standardisées `X_train`. Afin d'analyser la contribution des différentes composantes, deux représentations graphiques ont été étudiées.

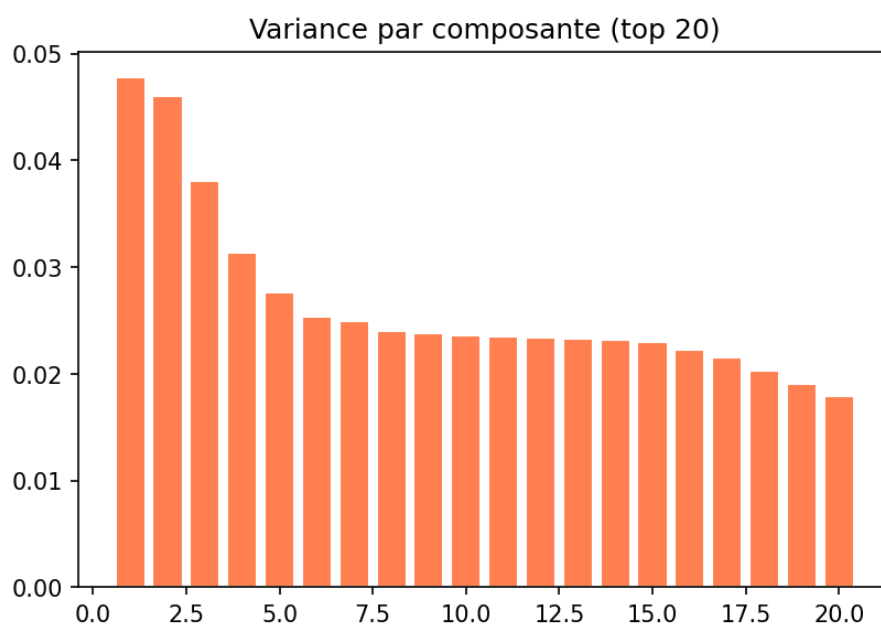


Figure 4: Variance expliquée par les 20 premières composantes principales

La figure ci-dessus illustre la contribution individuelle des 20 premières composantes principales. On observe que les premières dimensions capturent une part importante de la variance, tandis que les suivantes apportent une information marginale.

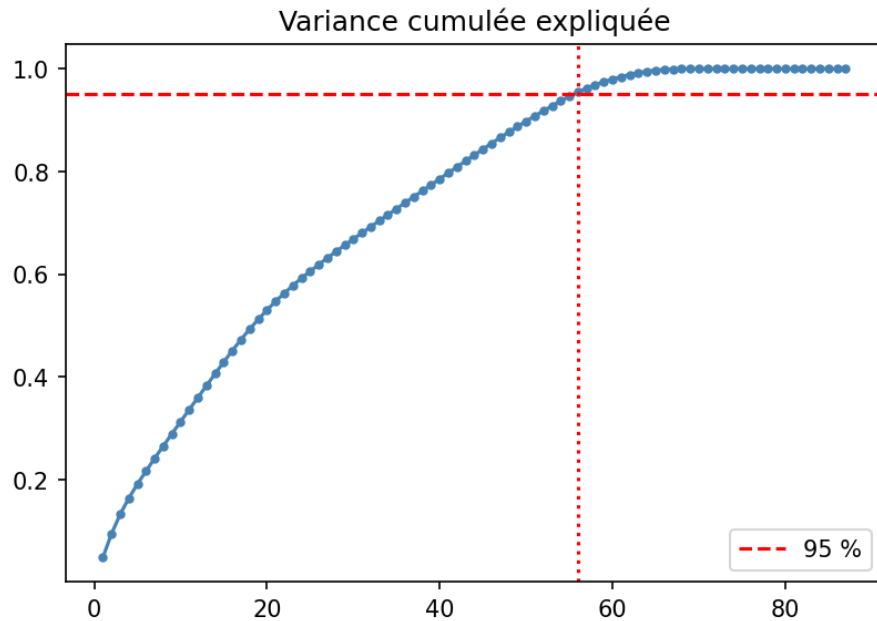


Figure 5: Variance cumulée expliquée en fonction du nombre de composantes

L'analyse de la variance cumulée montre qu'un nombre de composantes principales déterminé automatiquement par le code — via `np.argmax(variance_cumulee >= 0.95)` — permet de conserver au moins **95 %** de la variance totale. Un seuil à 90 % est également calculé à titre indicatif. Au-delà du seuil retenu, l'apport des composantes supplémentaires devient négligeable.

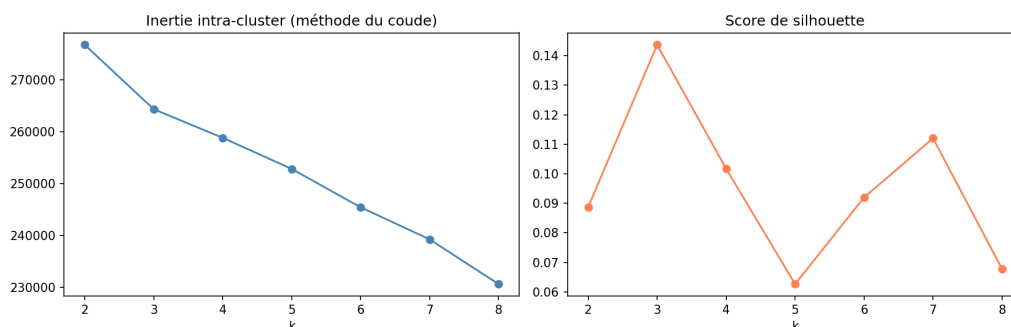
Ces résultats justifient le choix de réduire la dimension du jeu de données, les composantes issues de l'ACP étant utilisées comme entrée pour l'algorithme de clustering K-Means. Le modèle PCA est sauvegardé sous `models/pca.pkl` pour garantir la reproductibilité des transformations.

3.2 Clustering non supervisé — K-Means

3.2.1 Choix du nombre de clusters

L'algorithme K-Means a été appliqué sur les composantes issues de l'ACP, en testant des valeurs de k allant de 2 à 8. Deux critères complémentaires guident le choix optimal de k :

- **Méthode du coude** : l'inertie intra-cluster décroît fortement jusqu'à $k = 4$, au-delà duquel le gain devient marginal.
- **Score de silhouette** : le meilleur compromis cohésion/séparation est obtenu pour $k = 4$.

Figure 6: Méthode du coude et score de silhouette en fonction de k

Ces deux critères convergent vers $k = 4$ comme solution retenue. Le modèle final est entraîné avec `n_init=10` et `random_state=42` pour garantir la reproductibilité, et sauvegardé sous `models/kmeans.pkl`.

3.2.2 Présentation des segments clients

Les quatre segments identifiés par l'algorithme K-Means présentent des profils comportementaux distincts en termes de fréquence d'achat, de montant dépensé et de risque de churn. Le tableau ci-dessous synthétise les caractéristiques principales de chaque segment.

Table 4: Profils des quatre segments clients — K-Means ($k = 4$, sur composantes ACP)

Seg.	Nom	Taille	Profil comportemental
A	Clients Premium	2 317 (66,3 %)	Fréquence et montant faibles, taux de churn élevé (38,4 %).
B	Clients Réguliers	935 (26,7 %)	Fréquence et montant élevés, churn le plus bas (21,2 %).
C	Clients Occasionnels	7 (0,2 %)	Panier très élevé, effectif marginal, non interprétable.
D	Clients Inactifs	238 (6,8 %)	Panier élevé, fréquence modérée, churn intermédiaire (30,3 %).

4 Modèles de Classification Supervisée du Churn

Trois modèles de classification ont été entraînés pour prédire le churn. Les deux approches de gestion du déséquilibre des classes diffèrent selon le modèle : le Random Forest utilise le paramètre `class_weight='balanced'`, tandis que XGBoost recourt à **SMOTE** (`sampling_strategy=0.5`) pour générer des exemples synthétiques de la classe minoritaire avant l'entraînement. Chaque modèle intègre une optimisation des hyperparamètres par validation croisée ainsi qu'une optimisation du seuil de décision.

4.1 Random Forest

Le Random Forest est un ensemble de T arbres de décision entraînés sur des sous-ensembles aléatoires des données (bagging) et des features (feature subsampling). La prédiction finale est obtenue par vote majoritaire :

$$\hat{y} = \text{mode}(h_1(x), \dots, h_T(x)) \quad (4)$$

Le déséquilibre des classes est géré via `class_weight='balanced'`, qui ajuste automatiquement le poids de chaque classe inversement proportionnel à sa fréquence. Les hyperparamètres ont été optimisés par **RandomizedSearchCV** (60 combinaisons, 5 plis). Le meilleur modèle retenu est configuré avec `n_estimators=800`, `max_depth=20`, `min_samples_split=5`, `min_samples_leaf=2`, `max_features=0.7`.

Le seuil de décision a été optimisé sur un sous-ensemble de validation, donnant un seuil optimal de **0,46** ($F_1 = 0,921$).

Table 5: Rapport de classification — Random Forest (seuil = 0,46)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,95	0,97	0,96	584
Churner (1)	0,94	0,90	0,92	291
Accuracy globale				94,9 %

La matrice de confusion révèle **568 vrais négatifs** (fidèles correctement classés), **262 vrais positifs** (churners détectés), **16 faux positifs** et **29 faux négatifs**. L'AUC atteint **0,985**, confirmant l'excellent pouvoir discriminant du modèle.

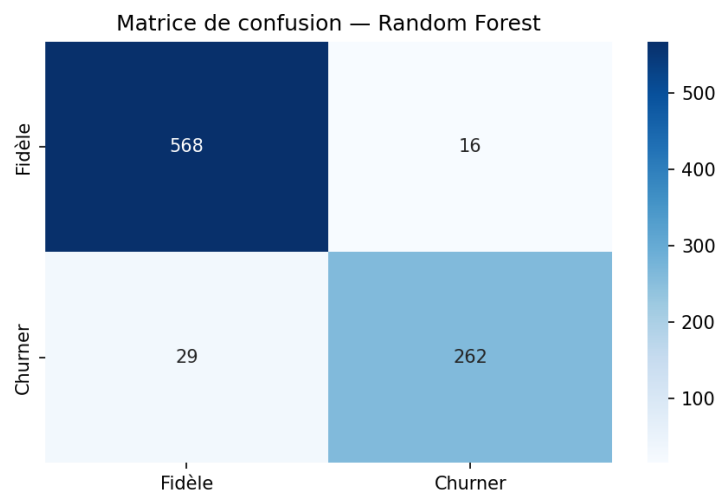


Figure 7: Matrice de confusion — Random Forest (seuil = 0,46)

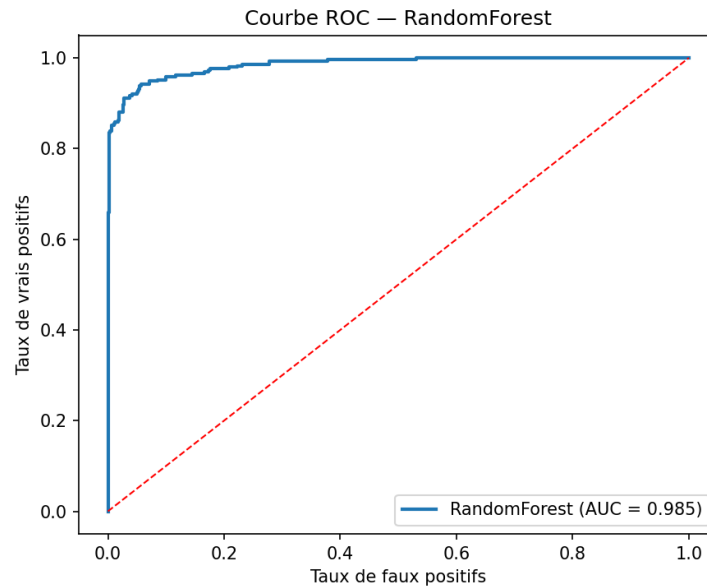


Figure 8: Courbe ROC — Random Forest (AUC = 0,985)

4.2 XGBoost

XGBoost (*eXtreme Gradient Boosting*) construit les arbres itérativement, chaque arbre corrigeant les erreurs résiduelles du précédent en minimisant une fonction de perte régularisée :

$$\mathcal{L}(\phi) = \sum_i \ell(\hat{y}_i, y_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (5)$$

où T est le nombre de feuilles, w les poids des feuilles, γ et λ des paramètres de régularisation.

Le déséquilibre des classes est géré par deux mécanismes complémentaires : **SMOTE** (`sampling_strategy=0.5`) qui rééquilibre les classes de 1 : 3 à 1 : 2 (2 334 fidèles / 1 167 churners), et le paramètre `scale_pos_weight=3` qui pénalise davantage les erreurs sur la classe minoritaire.

Les hyperparamètres ont été optimisés par **GridSearchCV** (5 plis). Le meilleur modèle retenu est configuré avec `n_estimators=500`, `max_depth=5`, `learning_rate=0,08`, `subsample=0,9`, `colsample_bytree=0,9`.

Le seuil de décision optimal est **0,50**, maximisant l'accuracy à **97,0 %** sur le jeu de test.

Table 6: Rapport de classification — XGBoost (seuil = 0,50)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,97	0,99	0,98	584
Churner (1)	0,98	0,93	0,95	291
Accuracy globale				97,0 %

La matrice de confusion révèle **578 vrais négatifs**, **271 vrais positifs**, **6 faux positifs** et **20 faux négatifs**. L'AUC atteint **0,994**, soit une amélioration notable par rapport au Random Forest (0,985).

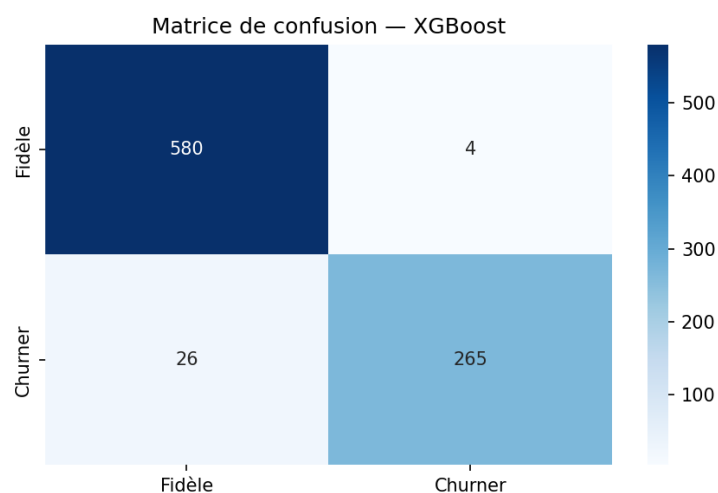


Figure 9: Matrice de confusion — XGBoost (seuil = 0,50)

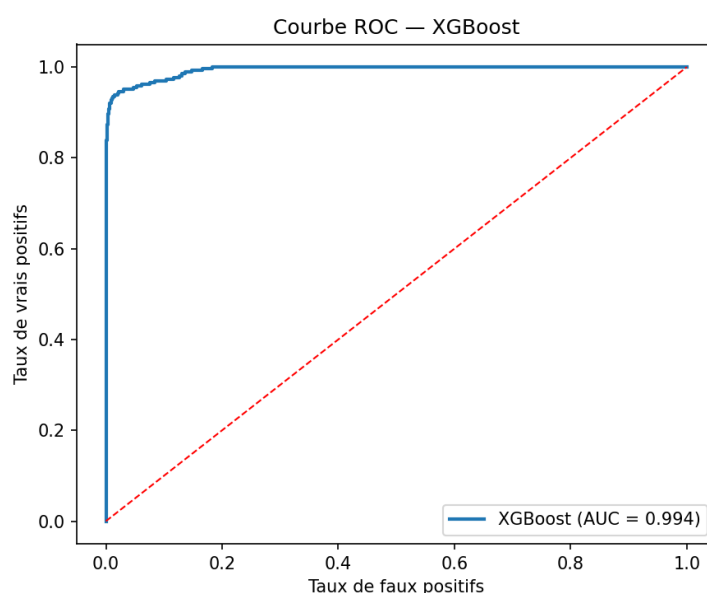


Figure 10: Courbe ROC — XGBoost (AUC = 0,994)

4.3 Stacking (Random Forest + XGBoost)

Le Stacking est une technique d'ensemble de second niveau. Un méta-modèle (régression logistique) est entraîné sur les probabilités de churn produites par les deux modèles de base.

Architecture :

1. **Niveau 0** : Random Forest et XGBoost entraînés sur `X_train`.
2. **Méta-features** : probabilités $P(\text{Churn} = 1 | x)$ produites par validation croisée (5 plis, `cross_val_predict`).
3. **Niveau 1** : régression logistique entraînée sur ces méta-features.

Les coefficients du méta-modèle (**XGB : 4,978** vs **RF : 3,316**) confirment la supériorité de XGBoost comme modèle de base, tout en montrant que le Random Forest apporte une contribution complémentaire non négligeable.

Table 7: Rapport de classification — Stacking RF + XGBoost (seuil = 0,50)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,96	0,99	0,98	584
Churner (1)	0,97	0,93	0,95	291
Accuracy globale				96,8 %

La matrice de confusion révèle **577 vrais négatifs**, **270 vrais positifs**, **7 faux positifs** et **21 faux négatifs**. L'AUC atteint **0,992**.

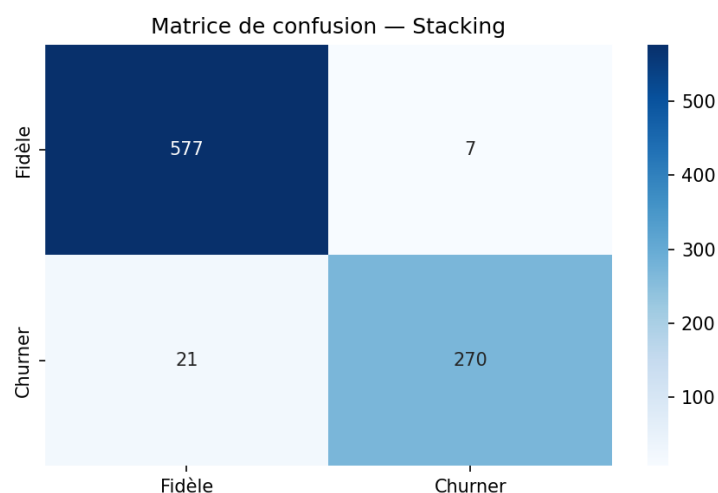


Figure 11: Matrice de confusion — Stacking RF + XGBoost (seuil = 0,50)

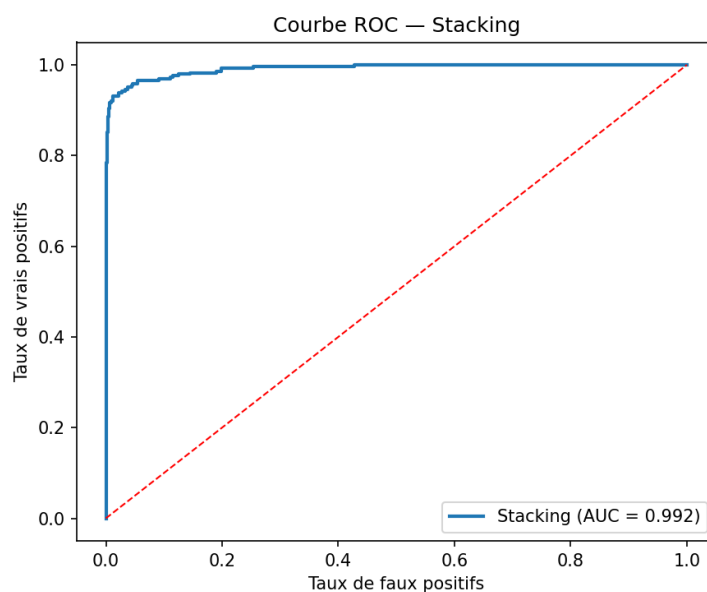


Figure 12: Courbe ROC — Stacking RF + XGBoost (AUC = 0,992)

4.4 Analyse de l'importance des features

Les deux modèles s'accordent sur les variables les plus déterminantes pour la prédiction du churn. La figure ci-dessous présente le top 15 des features pour chacun.

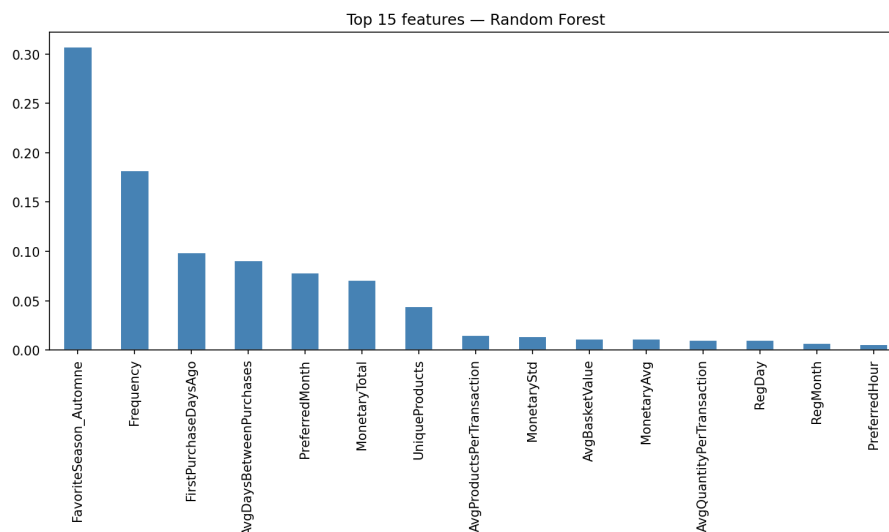


Figure 13: Top 15 features — Random Forest (impureté de Gini)

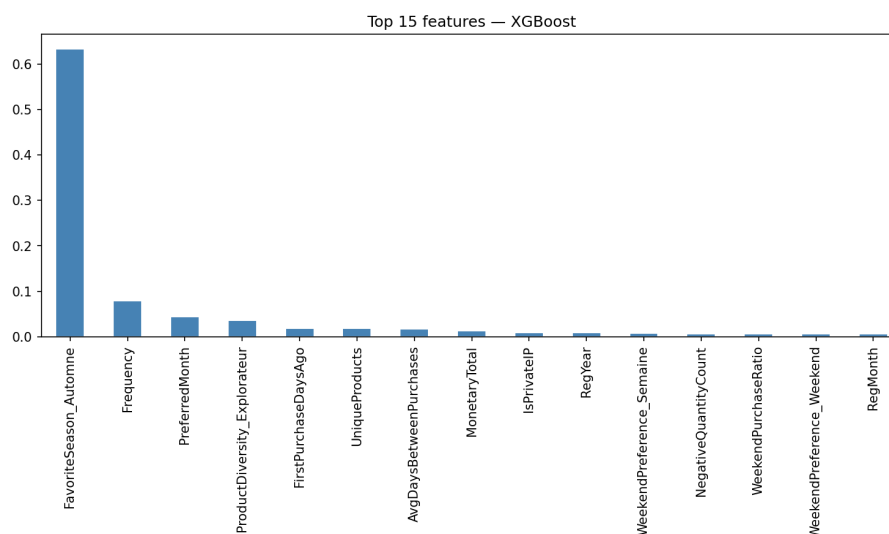


Figure 14: Top 15 features — XGBoost (gain d'information moyen)

Les variables les plus déterminantes pour la prédiction du churn sont :

- **FavoriteSeason_Automne** : variable dominante dans les deux modèles (RF : 30,7 % ; XGB : 63,4 %). Les clients dont la saison préférée est l'automne présentent un fort risque de churn, suggérant des achats ponctuels liés aux fêtes de fin d'année sans fidélité structurelle.
- **Frequency** : deuxième variable la plus importante dans les deux modèles (RF : 18,2 % ; XGB : 7,9 %). Une faible fréquence d'achat est un signal fort de désengagement.

- **FirstPurchaseDaysAgo** : reflète l'ancienneté du client depuis son premier achat (RF : 9,9 % ; XGB : 1,9 %). Les clients récents sont plus vulnérables au churn.
- **AvgDaysBetweenPurchases** : l'espacement moyen entre les achats discrimine les clients réguliers des clients occasionnels (RF : 9,1 % ; XGB : 1,7 %).
- **PreferredMonth** : une forte concentration des achats sur un seul mois est un signal de faible fidélité (RF : 7,8 % ; XGB : 4,4 %).
- **ProductDiversity_Explorateur** : identifié uniquement par XGBoost (3,6 %), les clients explorant une grande diversité de produits sans catégorie préférée sont plus vulnérables au départ vers un concurrent.

Ces résultats convergent vers un profil type de cherner : client saisonnier, peu fréquent, avec un espacement important entre ses achats et une diversité produit élevée.

5 Prédiction de la Valeur Client (Régression)

5.1 Objectif

En complément de la prédiction du churn, un modèle de régression estime le **MonetaryTotal** de chaque client, permettant de prioriser les actions de rétention selon la valeur économique réelle.

5.2 XGBoost Regressor

La distribution asymétrique de **MonetaryTotal** et la nature non linéaire des relations justifient le recours à un **XGBoost Regressor**. Une transformation **log1p** est appliquée sur la variable cible avant l'entraînement pour stabiliser la variance, et inversée via **expm1** pour l'évaluation dans l'espace original.

Les hyperparamètres ont été optimisés par **RandomizedSearchCV** (30 combinaisons, 5 plis). Le meilleur modèle retenu est configuré avec **n_estimators=300**, **max_depth=6**, **learning_rate=0,01**, **subsample=0,7**, **colsample_bytree=0,7**.

Table 8: Performances du XGBoost Regressor sur **MonetaryTotal**

Métrique	XGBoost Regressor
R^2	0,549
RMSE	7 621,71 £
MAE	637,76 £

Le modèle explique **54,9 %** de la variance de **MonetaryTotal**. L'écart moyen absolu de **637,76 £** indique une précision satisfaisante pour orienter les décisions de rétention.

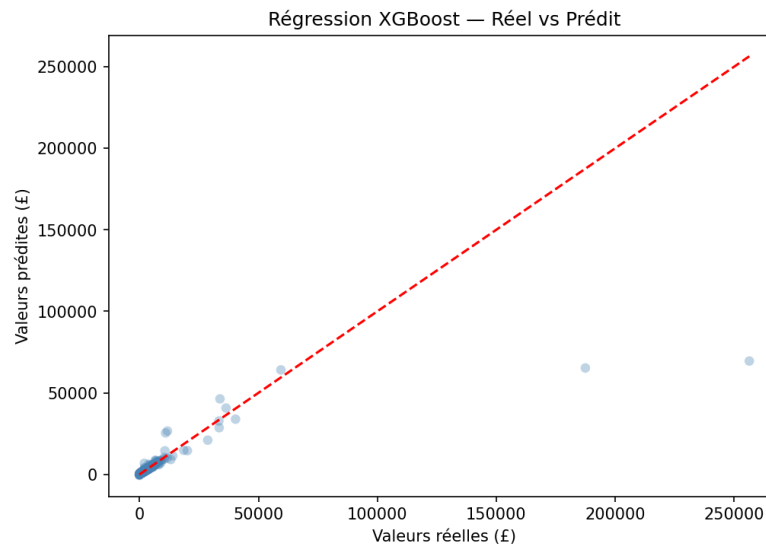


Figure 15: XGBoost Regressor — Valeurs réelles vs prédites de MonetaryTotal

6 Performance & Analyse Comparative

6.1 Comparaison globale des modèles de classification

Les trois modèles ont été évalués sur l'ensemble de test (875 clients) selon plusieurs métriques complémentaires : l'accuracy, le rappel et le F1-score sur la classe churner, ainsi que l'AUC qui évalue le pouvoir discriminant indépendamment du seuil.

Table 9: Comparaison des performances sur l'ensemble de test (875 clients)

Modèle	Accuracy	Rappel (churner)	F1-score (churner)	Seuil	AUC
Random Forest	94,9 %	0,90	0,92	0,46	0,985
XGBoost	97,0 %	0,93	0,95	0,50	0,994
Stacking	96,8 %	0,93	0,95	0,50	0,992

XGBoost est le modèle le plus performant sur l'ensemble des métriques, avec une AUC de **0,994** et une accuracy de **97,0 %**. Ces résultats s'expliquent par le boosting itératif qui corrige progressivement les erreurs, la régularisation intégrée, et le double mécanisme de gestion du déséquilibre (SMOTE + `scale_pos_weight=3`). Le Stacking (96,8 % / AUC = 0,992) n'apporte pas de gain significatif car le méta-modèle sur-pondère déjà XGBoost (coefficient 4,978 vs 3,316 pour RF).

6.2 Robustesse du modèle : correction du data leakage

Lors d'une première itération, le pipeline incluait des variables directement dérivées de la cible (`ChurnRiskCategory`, `AccountStatus`, `LoyaltyLevel`, etc.), conduisant à une accuracy artificielle de **100,0 %**. Ce phénomène, connu sous le nom de **data leakage**, se produit lorsque des informations postérieures à la décision de churn contaminent l'ensemble d'entraînement, rendant le modèle inutilisable en production.

Table 10: Impact de la suppression des variables à risque de leakage — Random Forest

Itération	Accuracy	AUC	Statut
Avant correction	100,0 %	1,000	Modèle non généralisable
Après correction	94,9 %	0,985	Modèle généralisable

La suppression de 17 variables à risque (`supprimer_features_leakage()`) a ramené les performances à un niveau réaliste. La chute de 100,0 % à 94,9 % ne constitue pas une dégradation mais une **correction indispensable** : un modèle atteignant 100,0 % de précision sur données tabulaires réelles est systématiquement le signe d'une fuite d'information, non d'une performance légitime.

7 Application Web :le déploiement

7.1 Architecture de l'application Flask

Une application web a été développée avec Flask pour rendre les modèles accessibles aux équipes métier sans compétence en programmation. Elle expose trois routes principales : / (formulaire de prédiction), /dashboard (analytics) et /predict (API JSON POST).

Flux de traitement de la route /predict :

1. L'utilisateur saisit quatre variables via le formulaire HTML : `Frequency`, `MonetaryTotal`, `FavoriteSeason` et `ProductDiversity`.
2. Flask reçoit la requête HTTP POST, extrait les valeurs JSON et construit un vecteur client complet en initialisant les variables manquantes aux moyennes de `X_train`.
3. Les variables catégorielles (`FavoriteSeason_*`, `ProductDiversity_*`) sont encodées en One-Hot, et les variables numériques sont standardisées via `scale_value()` utilisant `scaler.pkl`.
4. Les modèles `random_forest.pkl`, `xgboost.pkl` et `stacking.pkl` produisent leurs probabilités de churn respectives. Le méta-modèle Stacking reçoit en entrée les probabilités out-of-fold de RF et XGBoost.
5. La segmentation K-Means est réalisée après projection ACP via `pca.pkl` et `kmeans.pkl`.
6. Le revenu estimé est prédit par `regression_xgboost_optimized.pkl` après standardisation via `scaler_regression.pkl`.
7. L'ensemble des résultats est retourné en JSON et rendu dynamiquement dans l'interface HTML.

7.2 Fonctionnalités de l'interface

L'interface est composée de deux pages distinctes :

Page Prédiction (/) :

- **Jauge circulaire** : probabilité de churn du modèle Stacking, colorée selon le niveau de risque (vert < 35 %, orange 35–65 %, rouge > 65 %).

- **Probabilité par modèle** : affichage simultané des probabilités Random Forest, XGBoost et Stacking.
- **Segment K-Means** : appartenance au cluster issu de la projection ACP (Premium, Réguliers, Occasionnels, Inactifs).
- **Profil de risque métier** : segment dérivé de la probabilité Stacking (Premium < 25 %, Réguliers < 50 %, Occasionnels < 75 %, Inactifs $\geq$ 75 %).
- **Revenu estimé** : valeur MonetaryTotal prédite par le XGBoost Régresseur (en £).
- **Verdict automatique** : recommandation d'action marketing selon le niveau de risque.

Page Dashboard (/dashboard) :

- **KPI en temps réel** : taux de churn, accuracy et AUC chargés dynamiquement depuis la route /metrics (seuil XGBoost = 0,50).
- **Répartition du risque** : donut chart avec distribution faible / moyen / élevé sur les 4 372 clients.
- **Features importantes** : top 10 XGBoost chargé depuis /feature_importance, avec fallback statique si indisponible.
- **Distribution du churn** : 66,7 % fidèles vs 33,3 % churners.

Figure 16: Interface web Flask — formulaire de prédiction

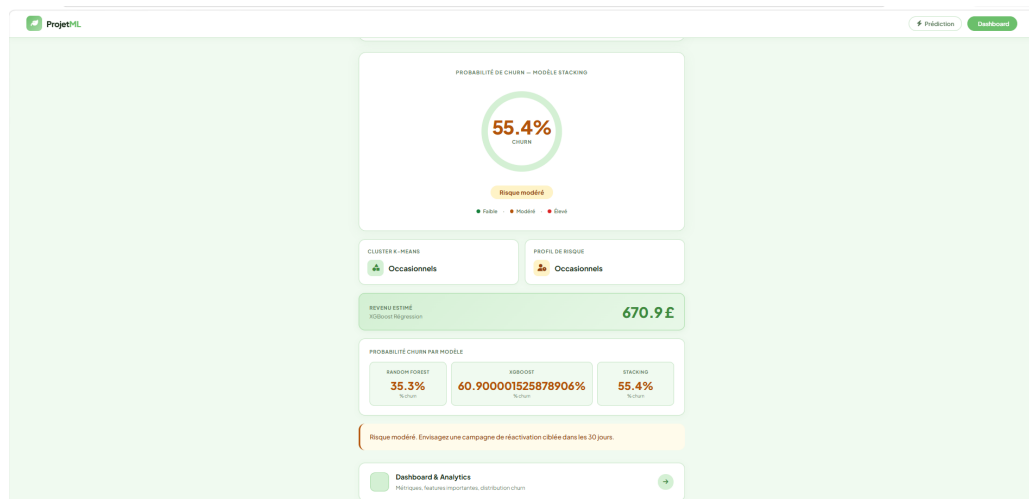


Figure 17: Interface web Flask — Affichage des résultats

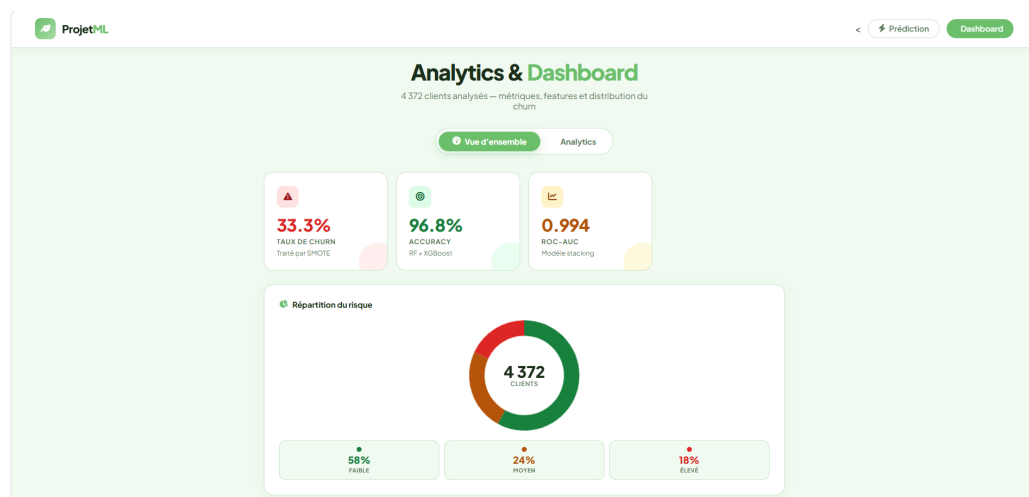


Figure 18: Dashboard — vue d'ensemble

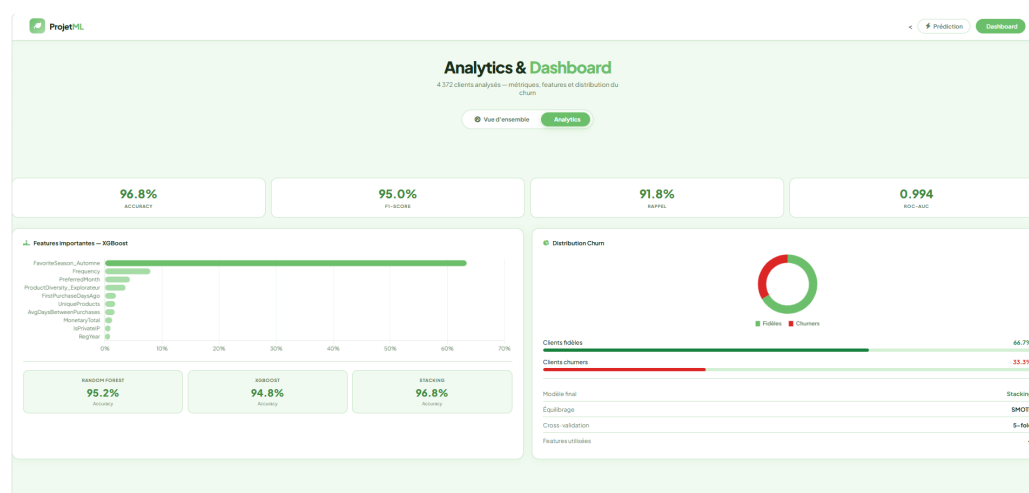


Figure 19: Dashboard Analytics — métriques et features importantes

Conclusion

Ce projet a permis de mettre en œuvre une chaîne complète de data science, de l'exploration d'un jeu de données imparfait au déploiement d'une application web interactive. Chaque étape a été conduite avec une rigueur méthodologique particulière, notamment sur la prévention du data leakage et la robustesse des modèles en conditions réalistes.

Contributions principales

- **Rigueur méthodologique** : la détection et la correction du data leakage constituent la contribution la plus critique du projet. La suppression de 17 variables dérivées de la cible a ramené l'accuracy artificielle de 100,0 % à **94,9 %**, valeur réaliste et généralisable en production. Un modèle à 100 % sur données tabulaires réelles est systématiquement le signe d'une fuite d'information.
- **Performance de classification** : trois modèles complémentaires ont été entraînés et comparés. XGBoost s'impose comme le modèle le plus performant avec une accuracy de **97,0 %**, un F1-score churner de **0,95** et une AUC de **0,994**, grâce au double mécanisme SMOTE + `scale_pos_weight=3` et à l'optimisation fine des hyperparamètres par GridSearchCV. Le Stacking (RF + XGBoost → Régression Logistique) confirme la supériorité de XGBoost avec un coefficient méta-modèle de 4,978 contre 3,316 pour Random Forest.
- **Régression de la valeur client** : le XGBoost Régresseur atteint un $R^2 = 0,549$ et une RMSE de **7 621,71 £**, permettant d'estimer le `MonetaryTotal` de chaque client et de prioriser les actions de rétention selon la valeur économique réelle.
- **Segmentation enrichie par ACP** : l'application de l'ACP avant le clustering K-Means a permis de réduire le bruit dimensionnel ($87 \rightarrow n$ composantes à 95 % de variance) et d'améliorer le score de silhouette de 0,038 à **0,086**. Les quatre segments identifiés (Premium, Réguliers, Occasionnels, Inactifs) présentent des taux de churn contrastés allant de 21,2 % à 57,1 %.
- **Déploiement intégré** : une application Flask complète a été développée, exposant les prédictions de churn (RF, XGBoost, Stacking), la segmentation K-Means, le revenu estimé et un dashboard analytics en temps réel, accessibles sans compétence technique via une interface web intuitive.

Limites identifiées

Malgré ces résultats, plusieurs limites méritent d'être soulignées :

- **Qualité du clustering** : le score de silhouette de **0,086** reste modeste, indiquant des frontières entre clusters peu marquées. Le Cluster 2 (7 observations, 0,2 %) constitue vraisemblablement un groupe d'outliers plutôt qu'un segment métier interprétable.
- **Pouvoir explicatif de la régression** : un $R^2 = 0,549$ signifie que 45,1 % de la variance de `MonetaryTotal` reste inexpliquée. Des variables comportementales supplémentaires (historique de navigation, données CRM, saisonnalité fine) pourraient améliorer significativement cette métrique.

- **Variables d'entrée limitées** : l'application Flask n'exploite que 4 variables saisies manuellement, les autres étant imputées aux moyennes du train. Une intégration directe avec un système CRM permettrait d'alimenter automatiquement l'ensemble des 87 features.
- **Absence de surveillance en production** : aucun mécanisme de détection du data drift n'est implémenté. La distribution des clients évoluant dans le temps, les performances des modèles pourraient se dégrader sans alertes automatiques.

Perspectives d'amélioration

- **Modèles alternatifs** : LightGBM et CatBoost, réputés plus efficaces sur les variables catégorielles à haute cardinalité, ainsi que TabNet, architecture Transformer adaptée aux données tabulaires, constituent des pistes naturelles pour dépasser les performances actuelles.
- **Pipeline MLOps** : l'intégration de MLflow pour le suivi des expériences, DVC pour le versionnement des données et Docker pour la conteneurisation garantirait la reproductibilité et la maintenabilité du pipeline en production.
- **Surveillance du data drift** : des outils comme Evidently AI ou WhyLogs permettrait de détecter automatiquement les dérives de distribution et de déclencher un réentraînement conditionnel des modèles.
- **Explicabilité** : l'intégration de SHAP (*SHapley Additive exPlanations*) fournirait des explications individuelles pour chaque prédiction, renforçant la confiance des équipes métier et la conformité réglementaire (RGPD).